



Enabling IoT platforms for social IoT applications: Vision, feature mapping, and challenges

Bilal Afzal^{a,*}, Muhammad Umair^a, Ghalib Asadullah Shah^a, Ejaz Ahmed^b

^a Sultan Qaboos IT Chair, University of Engineering and Technology, Lahore, Pakistan

^b Centre for Mobile Cloud Computing Research (C4MCCR), University of Malaya, Malaysia

HIGHLIGHTS

- Performance metrics are identified to select suitable OS(s) for specific hardware platforms in SIoT application(s).
- An OS-to-hardware architectural feature-mapping is provided to facilitate the collaboration among IoT devices.
- Suitable hardware and software platforms with compatible features are suggested for various IoT & SIoT applications.
- A model OS architecture is proposed and its principle requirements are articulated to improve and enhance the interaction among IoT devices.
- Associated open research challenges in implementing the model OS architecture are also highlighted.

ARTICLE INFO

Article history:

Received 15 June 2017

Received in revised form 4 November 2017

Accepted 3 December 2017

Available online 13 December 2017

Keywords:

Social IoT

Operating systems

Microcontroller architecture

Embedded systems

Resource-constrained devices

ABSTRACT

Social IoT (SIoT) is an emerging paradigm of IoT in which different IoT devices interact and establish relationships with each other to achieve a common goal. In essence, SIoT adapts a service-oriented architecture where heterogeneous IoT devices can offer or request autonomous services and collaborate on behalf of their owners. Operating Systems (OSs) are employed in IoT devices as they offer portability, threading support and access to development libraries; thus allowing easiness in IoT application development. Several OSs are available for IoT devices, but selecting an OS and hardware befitting for a particular IoT application is a critical task. In case of SIoT, the specific OS selection for hardware devices in various applications is even more challenging because of their collaborative nature. Existing surveys on OSs are mostly domain oriented and lack the discussion on hardware architectural features. As a consequence, it is infeasible for developers to choose best-suited OS for various hardware platforms which results in their underperformance in many application scenarios. This paper considers standard features of OS as well as hardware IoT platforms and provides an OS-to-hardware architectures features-mapping while exploring the unique requirements of SIoT applications. In doing so, resource-constrained IoT devices are particularly emphasized due to their memory constraints and power limitations. Further, a model OS architecture is proposed for devices in SIoT applications and associated open research challenges are identified. This research will benefit developers to best utilize IoT platform resources and to envisage an efficient OS for futuristic SIoT applications.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

Internet of Things (IoT) characterizes the interconnection of different devices with ubiquitous accessibility and built-in intelligence. IoT is already reshaping the way we interact with devices and providing us with the novel networking and socializing capabilities through intermediary devices. So much so that it will

soon transform our lives into a hyper-connected cyber-physical-social environment [1]. The potential devices include smartphones, bio-nano-things, body sensors, smart tags, wearable devices, embedded objects, and traditional electronic gadgets. These heterogeneous IoT devices have distinct connectivity and interfacing demands as well as service quality requirements [2]. Within the context of Social Internet of Things (SIoT), which is a specialized subset of IoT, the heterogeneous devices not only connect and interact but also socialize and collaborate with each other to achieve some specific task(s). This kind of social interaction requires cooperation among IoT devices which is the essence of SIoT. However, it cannot be truly realized unless built-in support

* Corresponding author.

E-mail addresses: bilal.afzal@kics.edu.pk (B. Afzal), umair@kics.edu.pk (M. Umair), ghalib@kics.edu.pk (G.A. Shah), ejazahmed@ieee.org (E. Ahmed).

is provided inside the operating systems (OSs), device firmware, or other software running on these IoT devices [3].

Specifically, an Operating System (OS) enables ease of device usage by offering portability, threading support and access to online development libraries. Thus OSs can facilitate better design and implementation of innovative social IoT applications [4]. Moreover, IoT developers mostly prefer to use an OS on IoT device(s), even in a single application entity, as it enables ease of application development along with other handy and extensible features. Particularly in SIoT application scenarios, these IoT devices are often heterogeneous and equipped with a variety of hardware architectures; hence they should have the necessary capability to efficiently run the potential OSs. Though, the choice of suitable OS for a specific hardware device in IoT application is usually a critical challenge. In case of resource-constrained environments, the desire for optimal selection of software (e.g., OS) as well as hardware (of specific type) resources become even more crucial to support dynamic social collaboration among devices in IoT applications.

An appropriate selection of OS for a unique hardware will result in rapid development of time-efficient and user-friendly IoT applications. It will particularly facilitate the inter-device communication within SIoT applications to improve the performance of interacting devices and efficacy of service delivery. Further, to allow interaction not only among devices with similar hardware architectures but also with dissimilar ones, OSs should incorporate the corresponding enabling features as well. For instance, the implementation of standardized communications interfaces within OSs ensure the efficient device collaboration and hence reduce the intermediary commute delays. Moreover, an OS can also play its role in improving the utilization of various resources, ranging from hardware computational resources to software resources along with other resources such as a battery and storage. Thus, OSs can act as enabling platforms for creating a cooperative ecosystem of all heterogeneous yet interactive devices in the future hyper-connected social environments.

However, existing literature does not take into consideration the functionalities of OS in tandem with hardware architectural capabilities. Although, it is inevitable for the efficient development of many IoT applications, especially where extensive social collaboration is involved. In fact, most of the research studies on IoT software and/or hardware platforms have overseen this essential requirement of matching the OS architectures to that of hardware architectures, which may lead to the underperformance of both, especially in constrained environments.

Specifically, existing research works and surveys lack any discussion on the interaction of OS kernel with system hardware. Several research papers have compared the OSs by their architectures, structure, operation and resource management but these comparisons are restricted to a specific domain [5–7]. Consequently, most of the hardware capabilities of IoT devices remain underutilized due to the selection of an inappropriate OS. There is a fair chance of choosing an inapt OS for a particular application or platform. This may also result in an OS selection with an entirely incompatible *communication stack* at most, or a mismatched one at least for IoT devices. Hence, such an OS selection may give rise to an under or over utilization of hardware and software resources which resultantly leads to an inability to fulfill the fundamental requirements of any IoT system. Also, an unfitting OS selection may result in bugs and deadlocks in IoT systems that can slow down the development cycle. Moreover, the existing research works do not consider the application-oriented feature classification of various OSs in IoT perspective. The research gap is further augmented when social interaction among devices comes into place. In short, the current research papers does not discern unique features of OS and hardware IoT platforms simultaneously for IoT and SIoT applications.

This paper strives to minimize all the above-mentioned discrepancies in the existing literature and bridges the research gap in the social IoT context. To begin with, it introduces several performance metrics which are taken into consideration while selecting an OS for any hardware platform. It then discusses the readily used OSs for different hardware architectures which are common in IoT devices. The main focus is on highlighting unique features of both OS and hardware platforms to enhance their performance and to eventually facilitate device-level interaction in IoT applications. The hardware capabilities supporting the performance metrics are examined as well. Further, the paper provides an OS-to-hardware architectures feature-mapping to facilitate the collaboration among IoT devices. Consequently, it suggests hardware and software platforms with compatible features for various SIoT applications. Last but not least, the paper proposes a model OS and its key architectural features while highlighting the principle requirements for improving the interaction and socialization activities among IoT devices.

Rest of the paper is organized as follows. In Section 2, we discuss the motivation for providing the mapping between OSs and hardware architectural features. Section 3 presents a comprehensive overview of existing literature. Several use cases of SIoT applications are discussed in Section 4 and considering these use cases, Section 5 discusses the different architectures of hardware IoT platforms and recommends OSs for them while presenting a comparative study in the context of social IoT. Subsequently, in Section 6 we suggest suitable OS and hardware platforms based on their features for different IoT applications. It also proposes the essential requirements for a model OS to better support the social IoT applications. Finally, open research challenges are discussed in Section 7 and Section 8 concludes the paper.

2. Motivation

Most of the IoT applications constitute resource constrained (RC) devices which have smaller footprints and power limitations. Owing to their fewer computational capabilities, these RC devices save us space as well as money since only specific and necessary hardware features are embedded in them which are crucial for a specific IoT application scenario. Considering the unique obligations of SIoT, the efficient interaction between RC IoT devices is essential to optimally utilize their limited software and hardware resources. Thus, the hardware architecture of any RC IoT device on which a particular OS is being used must fulfill certain requirements to run the OS efficiently. This proposition is equally valid conversely since an OS should also have necessary capabilities to fully exploit the hardware resources of any IoT device.

In other words, the conformance between the OS and hardware architecture of RC IoT device is imperative for optimal exploitation of their resources. For example, an OS performing complex *encryption functions* at transport layer to ensure trustworthiness among devices in SIoT applications must run on a hardware having enough computation power. Another critical OS feature is *Memory Access Time* which can be decreased by running an OS on a platform which implements fast register switching [8]. Similarly, hardware-based *Memory Access Scheduling* is an alternative method to reduce access time on hardware platforms [9]. This decrease in *Memory Access Time* and improvements in *Scheduling processes* both help in dealing with latency especially in mission-critical application scenarios of SIoT. Similarly, in Real Time OS (RTOS), task synchronization is a necessary feature which can be achieved by excessive *Inter-Process Communication*. Accordingly, fast interrupt response in hardware can help to achieve better task synchronization [10]. This kind of task synchronization is particularly crucial in a social SIoT application scenario where vehicles are communicating traffic information with each other [11]. Likewise, many other OS

features can be better utilized if hardware platform running these OS have enough capabilities to execute and compliment the OS functionalities; i.e. we can pick an OS with a smaller footprint for the hardware having *High Code Density* which will allow a very powerful OS to run on hardware with a tiny footprint. Considering this rhetoric, we believe that all these standard features specific to OSs together with hardware architectural capabilities of IoT devices (specially RC devices), can turn out to be the defining criteria in the development of multifaceted SIoT applications.

In a nutshell, this research is an effort to efficiently match the inherent capabilities of the hardware and software IoT platforms for both existing and future SIoT applications – in particular, and IoT applications – in general.

3. Related work

Our first objective is to conduct a joint survey of OSs and hardware architecture which are frequently utilized in RC devices or which are notably employed for sensor networks. Secondly, we investigate the research studies that, either simultaneously or separately, discuss the OS and hardware platforms used in constrained IoT devices. Later on, the discussion is extended to devices in SIoT applications while considering the unique requirements of interactive relationships established among them.

The currently available research work in the literature on OSs can be broadly classified into two major areas including Wireless Sensor Networks (WSNs) and IoT.

3.1. WSNs

In last decade or so, many research studies are conducted on the suitability of OSs for constrained environments [12–15]. The design of a lightweight Linux Security Module (LSM) is presented in [16] which claims implementation with minimal changes in the kernel. The LSM module design ensures generality and its access control for other modules. In [17], the performance analysis of tailored Linux OS on different microcontrollers is provided. The paper discusses the memory management and processes running in the tailored Linux design while also describing some key differences in the kernel with that of Desktop Linux. While the majority of other survey papers on constrained OSs are domain oriented as they describe features and layers of OSs which are appropriate for particular domains. Primarily these surveys are focused on WSNs [18–25].

For instance, a research paper in [18] has discussed the support for different programming languages framework along with the networking protocols supported by various OSs for WSNs. The hardware platforms on which these OSs can be ported are also figured out. Authors in [21] discussed multiple OSs while considering various features required for WSNs. These features are discussed in the light of architecture, programming model, scheduling techniques, memory protection and management, communication stack, resource sharing algorithms and real-time capabilities. This research work also considered the requirements and key challenges for any OS to run in domain-specific hardware in constrained environments. The software stack of the OSs are taken into consideration and availability of some core libraries in software development kit (SDK) is investigated as well.

In addition, several papers provide a detailed discussion of the modules or stacks having necessary capabilities for WSNs within a particular OS. A potential OS for WSNs namely RETOS (i.e., Resilient, Expandable and Threaded OS) is presented in [19] and [20]. RETOS, as the name suggests, provides system resiliency, dynamic configuration of the kernel and multi-threaded programming interface for different microsensor platforms. For performance evaluation, it is ported to AVR and CC2430 based sensor motes [20].

However, the performance comparisons of RETOS with existing OS for WSNs is not included. Another popular OS for resource-constrained sensor networks, namely Tiny OS, is proposed in [22]. It provides event-driven execution, flexible concurrency model, and component-oriented application design. The key features include low power and event-centric architecture suitable for sensor nodes in WSNs. Authors in [23] propose Contiki OS for WSNs which provides dynamic loading and offloading of modules for wireless sensor nodes. The kernel architecture of Contiki OS is described in detail, along with its scheduling mechanism, its power saving mode and the handling of various services. Besides, the communication stack of Contiki OS is also elaborated, and correspondingly, the code footprint in specific scenarios is approximated. In [24], a model architecture for RTOS is proposed and a real-time kernel (called FireKernel) is presented while focusing WSNs in rescue use-case scenarios. The FireKernel uses preemptive scheduling and also provides mutexes and synchronous message passing functionalities for inter-process communication. Similarly, a lightweight layered communication stack for WSNs, namely RIME, is proposed in [25] to reduce the implementation complexity. The objective of RIME architecture is to achieve less complexity and thinner layers while significantly reducing the footprint of communication stack, without compromise on the reliability.

A handful of comparative studies on OSs is also offered by existing literature [6,7]. Research on this front comprises the analysis of the functionalities of different OSs, their architecture, structure along with their operations and their recommendations for WSNs applications. However, these studies put little or no emphasis on the hardware platforms capabilities running the OSs under discussion. The research works in [6] and [26] have exclusively presented the OSs performance on mobile-phone platforms. Specifically, in [6], different OS environments are examined, and their strengths and weaknesses are highlighted. Further, the implementation and performance aspects of the OSs under discussion are also considered. While authors in [26] presented a research study on Android and iOS for accessing Internet streaming services while comparing these OSs in various fractions. The hardware dependencies, security and other characteristics related to the memory management have been extensively discussed in [26]. The uniqueness of their approach is to analyze the effect of mobility on the performance of OSs.

3.2. IoT

The majority of research works on the OS selection specific to IoT domain are either focused only on a certain OS or have limited scope [7], [27–36]. For instance, the technical aspects of various OSs are individually considered in several papers [27,28]. Some major concerns in the design of OSs for typical network domains are highlighted. In this context, the architecture of these OSs is elaborated, and corresponding advantages and disadvantages due to the underlying architecture for these domains are reviewed in these papers. The algorithms developed for resource sharing, memory protection, and management of these OSs are also discussed. Moreover, these papers describe design and implementation of typical modules/layers in specific OSs. Similarly, features of specific layers or modules of OS to serve in a certain domain are also emphasized in some research studies.

Specifically, a comparative survey on existing OSs for IoT is conducted in [7]. Therein, authors comprehensively discussed all the potential OSs for IoT and compared them according to a specific set of criteria. In [29], a short survey on the selection of suitable OS for two different classes of IoT platforms is presented. The survey briefly discusses the requirement of choosing the right OSs for IoT hardware platform architectures including the Cortex-M and TI MSP430 microcontrollers. Another brief survey on the different

available options of OSs for IoT devices is presented in [30]. The paper discusses the desired features of an OS for IoT while presenting a comparison of existing OSs based on these features and a generic framework for designing a potential OS for IoT. We have opted a similar approach, but in our case, the hardware architectural features of IoT devices are also highlighted while suggesting specific OS for IoT device(s). In addition, the application specific scenarios where device-level interaction and socialization become the key requirements of SIoT are also particularly emphasized.

Similarly, authors in [32] discussed the various aspects of OSs for IoT environments while briefly describing the supported platforms. The research work in [33] presented the RIOT as a potential OS for IoT. It examines the design aspects and architecture of the RIOT OS and also compares its features with that of the other OSs which are commonly used for IoT. Another potential OS proposed specifically for IoT is mbed OS which has a modular architecture. The rich communication stack in mbed OS contains different protocols which are specifically proposed for IoT such as MQTT, TLS, 6LoWPAN, etc. In [34], authors gave the vision of generic architecture of RTOS for IoT and the associated challenges are also mentioned. However, the scope of all these research works, [29–34], is limited and the specific features of the OSs under discussion are not mapped to that of the hardware architectural capabilities of IoT devices.

To sum up, the extensive literature overview presented above considers several different kinds of OSs for their suitability and applicability for WSNs and IoT within constrained as well as unconstrained environments. However, these research studies are either domain oriented or they only consider one or more OS for a specific hardware. None of these research studies comprehensively discuss the distinct features of OSs along with their potential suitability for commonly used IoT hardware platforms, and more specifically for resource-constrained devices.

Quite recently, a survey paper in [35] draws attention to the different hardware platform architectures used in IoT. Another interesting research work on OSs for low-power IoT devices regarding enabling technologies, standards, and requirements is presented in [36]. Our approach is somewhat similar to these papers as they discuss the hardware architectures of devices on which different OSs for IoT are ported. Likewise, in [15] an alternate but analogous approach is adopted for WSNs wherein authors provides a mapping of different applications requirements to existing OSs. However, the key aspect lacking in these studies (and in fact all the research works cited in this paper) is the mapping of OS features on that of hardware architectural capabilities. The feature-mapping provided in our case is expanded to different IoT applications to select most appropriate OSs along with the optimal hardware for the corresponding application specific scenarios while considering their unique service and interfacing requirements. Moreover, the OS-to-hardware architectures feature-mapping advocated in this paper is principally focused on the socialization aspect of IoT applications to facilitate superior interaction among heterogeneous kinds of devices. For this reason, we review the current literature on social IoT in the subsequent part of this Section.

3.3. Social IoT

An emerging subset of IoT is SIoT paradigm which addresses the establishment of social relationships among interacting objects. It can have numerous applications in hyper-connected atmospheres such as IoT-enabled smart cities, smart communities, smart grids, telemedicine, etc. SIoT addresses the relationships establishment due to the interaction of IoT devices with each other as well as their users over a period. The fundamental concept is that the devices in IoT applications provide a different kind of integrated services to achieve some common goal and to be mutually beneficial. SIoT has

attracted the significant interest of research community in last few years [37,38]. The research studies have assessed the interaction among devices, the resulting relationships, and composite services along with the related aspects of trustworthiness. The potentials of introducing social facet to provide integrated networking services and to support novel IoT applications are also explored.

The first noteworthy step to converge IoT with social networks is taken back in 2010 [39], wherein the authors proposed that the individuals can share the services offered by the smart objects owned by them with that of their friend's objects. The basic idea is to utilize human social networks (e.g. Twitter) as service discovery and provisioning infrastructure. However, the proposed notion does not align with the fundamental concept of IoT in which the ubiquitous connectivity of object is envisioned to provide services to humans. Another attempt is made in [40] where authors discussed the integration of IoT with social networks and conceived the term social IoT. An important step in laying down the vision of SIoT is taken in [41]. Therein, the various policies to determine the establishment and management of relationships among IoT objects are discussed. An architecture for integrating the social networks with the IoT is presented, and the SIoT network characteristics are analyzed through simulations. A three-layered model composed of sensing, network and application layers is followed to propose a system architecture for SIoT [42].

A fundamental component of a SIoT system is *relationship management*. In this context, a proposition of different kinds of relationships in which SIoT objects can be engaged-in is presented in [41]. Four basic relationships that can be established between interacting objects include parental–child relation, co-location/co-work based relation, object ownership relation, and social object relation. Further, these relationships are utilized to enhance *network navigability to discover* and provide *on-demand services*, similar to the notion of human beings looking for establishing relations and friendships [43,44]. However, the relationship management functionality is in turn dependent upon the activities and features (such as device type, vendor, battery life, compute power) of interacting objects within an IoT application. After identifying objects potentials, only then it can be known that which type of IoT objects support and fulfill corresponding SIoT components and their associated requirements. In addition, the software selection that precisely matches with the capabilities of the object is another important decision to be made. An insightful study of the hardware architectural capabilities of IoT devices to be involved in SIoT applications along with their software capabilities is therefore inevitable.

In addition, apart from *relationship management* and *service discovery*, other key functionalities of a social IoT system include *service composition* and *trust management* which are emphasized in recent research studies on SIoT [41,42]. Trust management is one of the vital facet of the SIoT system and equally important to realize the SIoT paradigm. Research in [44] particularly focuses on the trust management aspect of the SIoT. It highlighted the significance of trust establishment in deciding the relationship and collaboration among the IoT objects. Similarly, a trust management protocol for social IoT system is proposed in [45] where the trust parameters settings can be chosen adaptively depending upon the change in social conditions. Such solutions can perform more efficiently by integrating them within the suitable OS while also matching with the known hardware features of participating devices. For instance, by incorporating the trust management protocol inside an appropriate OS will not only facilitate the accurate assessment of trust but also maximize the performance of the concerned application. Similarly, we can exploit the hardware and software resources of IoT objects and their inter-relationships to create and provide new composite services [42,44,46]. To harness the desired outcome, again the software or OS deriving the functionalities of

SIoT devices should be capable enough to meet the associated interaction requirements [47].

While the authors in [48] proposed and implemented an experimental platform specific to SIoT, however, they did not emphasize the significance of OSs for device interaction in SIoT applications. In fact, there exists no prior discussion on the hardware and software (OS) platforms capabilities and associated requirements of various SIoT applications in the current literature. To the best of our knowledge, this paper is a first effort to provide an OS-to-hardware architectures feature-mapping for improving the interaction and collaboration among IoT devices. The proposed feature-mapping is therefore compliant for SIoT applications in particular, and for other multidisciplinary IoT applications, in general.

4. Use cases of social IoT

In this section, we present several SIoT use cases with *real-life IoT devices* that are engaged in some socialization activities. Several SIoT application scenarios can emerge involving the continuous and evolving interaction among a variety of devices. Further, various type of relationships can be established among devices according to the context and nature of their interactions [49]. Some of the prospective socialization opportunities in IoT use cases and applications are as follows.

4.1. Smart shopping mart retailing

A customer enters in a shopping mart premises and turns on her *smartphone*. The smartphone integrates with and shows the shopping mart App which, after authenticating the customer from a *smart camera* deployed at the mart entrance, generates her personal user profile. The smartphone App interacts with a *refrigerator* at home to provide a check-list of required grocery items. It then collaborates with the *local server machine* to find and notify the location of these items to the customer through *sensors* and *RFID tags*. Once the customer selects and pick up the items, the invoice is generated from *local server* on her smartphone and customer may then proceed with the amount payable using her online credit card.

4.2. Healthcare and telemedicine

Hospitals and healthcare centers enabled by IoT vision can provide better medical facilities to rural areas, and patients are monitored in real-time, their medical records at *cloud server* will be accessed with immediacy and treatment is suggested straight away. *Smart medicine-box* can remind patients their dosage and interact with *body sensors* to efficiently monitor and plan any change in the dosage [50]. At highways, *smart cameras* collaborate with each other to provide quality live streaming of an accidental event to a nearby hospital. On-road/air ambulance get instantly notified to evacuate the patient(s) in critical condition, while the *sensors* installed in the mobile health vehicles may collaborate with the *cloud server* and/or *body sensors* to instantly adjust the vehicle's medical apparatus parameters in the best manner according to the patient's medical history.

4.3. Intelligent multimedia streaming system

A user is traveling in a smart driverless car, and he wants to entertain himself by watching a movie. A virtual movie catalogue on car's *microHD screen* provides the latest recommendations anticipated by the discussion on trending movies among user's friends on social media. It may also collaborate with the personal belongings of the user such as his *iPhone*, and *laptop* or *multimedia system* at home, thus forming a personalized cloud. The cloud fetches and analyzes the historical meta-data to identify

various movies in which a user may be interested in, and display them on *microHD screen* through collaborative tethering services. Meanwhile, *smart sensors* installed in the car are managing driving parameters and controlling navigation via live maps. These *sensors* continuously monitor car's accelerator speed and collaborate with *microHD screen* to allow dynamic adjustment of the data rates and bandwidth of multimedia streaming according to the speed of the car to ensure high-end real-time experience.

4.4. Smart societal interaction

A smartphone App can be developed that learn about friendship of two individuals from social networks while they are roaming in a common leisure place. It can track their location through the collaboration of on-location *camera devices*, and notify them on their *smartphones* that they can potentially meet with each other while also suggesting them a suitable socializing venue. Alternatively, *camera devices* in a restaurant or a gym collaborate with each other to monitor the behavior of regular visitors (but at completely different timings). This information is stored at the IoT gateways and after learning from social networks about the visitors who are living in nearby vicinities, suggestions can be made on smartphones that they can socialize with each other.

4.5. Traffic surveillance and road safety

A smart traffic monitoring App can combine the information provided by connected *vehicles*, driver profiles, *road sensors* reporting occupancy patterns and disseminate this information through IoT *edge devices* in a real-time manner to reduce or avoid traffic congestion. Another possibility is that a roadside *surveillance camera* detects an ongoing theft situation in the street, generates an *alarm*, and may also provide video footage to alert the *mobile police vehicles* to take necessary action.

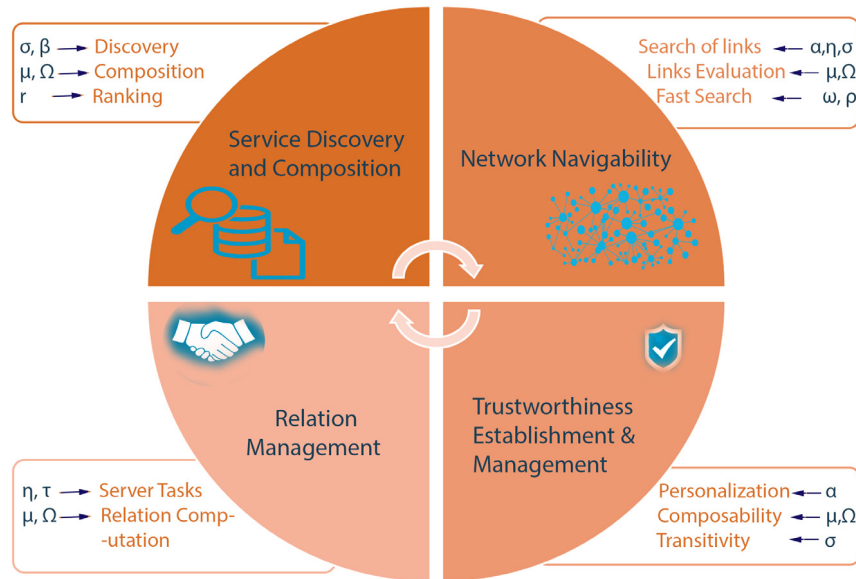
All of these example scenarios accentuate that by employing SIoT concept, the device-level interactions and collaborations can provide new socialization opportunities for different IoT devices as well as for their owners. It can contribute to identifying new information from ordinary data streams and may also facilitate in discovering knowledge out of it. Moreover, the various interacting devices mentioned above such as *smart cameras*, *sensors in the refrigerator*, *RFID tags*, *body sensors*, *smart medicine-box*, and even personal gadgets such as *smartphones*, *laptops* or *multimedia systems* can eventually develop relationships with each other based on the level of trustworthiness. The relationship establishment thus creates further possibilities such as an increase in network navigability, scalability, device as well as service discovery, service composition, etc., among other exciting possibilities.

However, these prospective SIoT possibilities can be realized and incorporated into IoT devices only if they exhibit the corresponding capability within their hardware and/or software components. For instance, the establishment of trustworthiness among devices in various SIoT use case scenarios is reliant on the computational capability of their hardware. Similarly, heterogeneous IoT devices can only interact and identify the relationship with each other if they have the rich type of communication interfaces available inside the hardware which in turn play a crucial role in extending the network scalability. However, any communication interface itself requires the matching support from the OS(s) driving the hardware both in the form of communication stack and protocols. In fact, the support of many other features of hardware architecture or the software program embedded within them is required for other SIoT components as well. Hence, the significance of key features of hardware architectures and that of OSs is recognizable to enable socializing possibilities among IoT devices and resultantly, the OS-to-hardware architecture feature-mapping turns out to be vital in achieving all these SIoT objectives.

Table 1

Performance metrics for choosing an OS along with feature-mapping symbols.

Features in OSs used as performance metrics	Corresponding feature in Hardware	Features mapping symbols
Memory access time	Memory access scheduling and register switching	α
Inter process communication	Fast interrupt response	β
Footprint	Code density and RAM size	ψ
Computational functions	Floating point unit and computational efficiency	μ
Execution speed	Processor and instruction pipelining	ω
Communication stack	Connectivity interfaces	σ
Power efficiency	Low power	η
Timers	Real-time clock	τ
RTOS	Deterministic response time	ρ
Robustness against bugs	Reliability	r
Encryption and decryption functions	Computational efficiency	Ω
Sensor interfacing algorithm	Powerful DSP Extension	χ

**Fig. 1.** Identification of feature-mapping symbols to basic components of SLoT network platforms.

5. OS recommendations for different hardware platforms

The constrained devices in SLoT applications have different requirements such as device-level interaction, socialization, trustworthiness evaluation, among others [38]. These requirements can be enabled through specific features in embedded OSs and hence should be explored prior to an OS selection. Moreover, to efficiently manage the hardware resources, the selected OS, in turn, needs to be run on suitable hardware architectures. In this Section, an OS-to-hardware architectures feature-mapping is provided so that the corresponding features meet the requirement of SLoT applications. Firstly, several unique features of embedded OSs are identified which are significant for SLoT paradigm (some of them are already discussed in Section 2). These features are taken as performance metrics for choosing an OS and stated along with their corresponding support capabilities desired for hardware platforms in Table 1.

Consequently, considering the unique requirements of SLoT, Fig. 1 associates each of the above mentioned feature-mapping symbols with various specialized components of SLoT which are identified in the seminal paper on SLoT [40].

The SLoT devices are heterogeneous and every device has to go through and perform different phases of friendship establishment with other devices to avail some service(s) or compose entirely new ones. So at each phase, the devices perform some type of ranking to evaluate and assess the available options according to different SLoT components. To make this ranking and evaluation easy and efficient, we provide a mapping of features of devices

(software and hardware resources) on different SLoT components. Thus, Fig. 1 encompasses the complete SLoT cycle of social relationship establishment starting from *service discovery* up to *relationship management*. For instance, considering the transitivity property in case of trustworthiness, a device can get more trust opinions from other devices if it is supported by rich communication interfaces (σ) offered by its OS and the hardware. In addition, to maintain a certain level of trustworthiness among devices, the better computational capability is an essential requirement which is represented by ' Ω ' feature-mapping symbol. As another example, the SLoT server which facilitates *relation management*, requires the real-time support (τ) in an energy efficient (η) manner while accomplishing server tasks. Similarly, to evaluate and compute the relations among SLoT devices, the feature-mapping symbols like ' μ ' and ' Ω ' are relevant. Moreover, a crucial task in *service composition* is ranking, and the devices having better computation capability, ' μ ', will show superior performance. *Network navigability* is computed and evaluated at runtime, and the concerned devices must have ' ω ' and ' ρ ' features support in their OS and hardware to facilitate navigation operations.

Therefore, considering the feature-mapping symbols provided in Table 1 and Fig. 1, the OSs having strong capabilities to improve the associated performance metrics are thus required to be recommended for the frequently used hardware architectures in SLoT applications. In this context, some of the most common and well-known hardware architectures used for constrained IoT devices are discussed in this Section ranging from basic Harvard 8051 to more

complex ones such as Advanced RISC Machine (ARM) architectures. The various characteristics of the hardware architectures are also mentioned afterward. Accordingly, one or more embedded OS is selected for these hardware architectures based on the features of OSs. Finally, the mapping of these OS features to hardware capabilities is provided at the end of this Section. We consider that the proposed mapping will allow better utilization of both the OS and hardware architectural resources in SIoT applications.

5.1. 8051 architecture

The 8051 is considered to be most basic architecture. It contains a Boolean processing engine which supports bit level computations. Another prominent feature is bank selectable working register set which results in faster context switching [51]. Other hardware capabilities of the 8051 architecture that should be considered while selecting suitable OS are specified as under:

- (1) Arithmetic and compare instructions.
- (2) Four register banks each containing 8 memory mapped registers.
- (3) Fast interrupt response is achieved via optional register bank switching.
- (4) Interrupts and threads with selectable priority.
- (5) Some 8051 derivatives also contain power saving mode.

Considering the hardware specifications of 8051 architecture, it is basically suitable for bare metal applications. However, there may be a need of multitasking in a few lightweight SIoT applications. Hence, following OSs are suitable for 8051 architectures considering performance metrics identified in Table 1.

5.1.1. RTOS

A real-time operating system (RTOS) provides four main functionalities including deterministic task scheduling; task synchronization mechanism; inter-process communication mechanisms; and timer services [10]. Hence, an RTOS can provide multitasking in several SIoT applications. Moreover, 8051 architecture can provide fast context switching due to its multiple register banks which are switchable with a single instruction. Hence, RTOS is a suitable option for IoT devices with basic 8051 architecture based microcontrollers.

5.1.2. TinyOS

TinyOS is a component-based OS whereby modules can be added to need basis making it suitable for IoT and other sensor network devices. It is a potential candidate for 8051 as it has memory constraints. Written in nesC¹ language, which optimized for the constrained sensor devices, it has a small footprint. TinyOS programs are built of software components, some of which use hardware abstraction layer. The components are linked to each other using interfaces. These modules are added to the user program as necessary, so that it may not overload processor and main memory; making it suitable for 8051 architecture.

5.2. PIC

Another renowned hardware architecture is named as Peripheral Interface Controller, i.e. PIC. It is an enhanced HARVARD architecture as it contains *separate data and program memory* and also *self-programmable flash memory*. Moreover, some families of PIC also support *in-circuit debugging*. Following are the significant characteristics of the PIC architecture that should be considered while selecting a suitable OS:

- (1) A small number of fixed-length instructions.
- (2) Rich single-cycle instruction set (2 clock cycles, or 4 clock cycles in 8-bit models), with one delay cycle on branches and skips.
- (3) A unique feature of PIC is hardware stack for storing return addresses.
- (4) It has a small amount of addressable data space. Minimum 1 byte is accessible in a single instruction.
- (5) The program counter is writeable and is contained in data space.
- (6) One accumulator (W0), the use of which (as source operand) is implied (i.e. is not encoded in the opcode).

Since PIC is an enhancement to 8051, the recommended OSs for PIC remain the same as that of 8051 architecture.

5.3. AVR

The Atmel's AVR is an advanced Harvard architecture in which there is isolation between program memory and data memory. In AVR, there is on-chip flash for program memory storage. Additionally, an AVR microprocessor has the in-system programming capability, which means it can be reprogrammed while they are connected to the system. It is basically a Reduced Instruction Set Computer (RISC) machine which signifies the compact combination or complexity of instructions. The compilers in RISC architectures can mostly find single-cycle instruction for an operation. The resulting machine code enhances speed while reducing the footprint of both SRAM and Flash. AVR has a two-stage single-level instruction pipeline which improves computation efficiency and less power consumption in sensor and IoT devices.² Other hardware capabilities of AVR architecture are listed as under:

- (1) Bi-directional general purpose I/O ports
- (2) Multiple internal Oscillators
- (3) In-circuit debugging
- (4) High code density
- (5) Brownout detection
- (6) Support for AES and DES

Considering these hardware specifications, following OSs are suitable for AVR based IoT hardware platforms:

5.3.1. Femto OS

The Femto OS is a very concise real-time preemptive operating system (RTOS) for embedded microcontrollers with minimal RAM and flash. It is easily portable due to its minimalism. It is specially designed for ATmega and ATtiny series. The implementation is in C containing the separate port file. The average footprint is 1k to 4k, while 10–20 bytes of RAM is required for OS and the processes take 6 bytes of RAM. So, overall requirement of RAM is 20 to 40 bytes including stacks for real-time response. The four RTOS tasks can be executed simultaneously. There is no isolated idle task and most API functions are called within OS space. Moreover, interrupts can be mapped on OS services. In short, a Femto OS is well suited for relatively powerful hardware architectures.³

5.3.2. Mbed

A prominent OS specifically designed for RC IoT devices is mbed. Its energy efficiency makes it suitable for constrained environments. The Mbed OS provides all the necessary features required for IoT environments such as efficient communication, security and device management. It has a modular architecture and contains a

¹ David Gay et al., The nesC Language: A Holistic Approach to Networked Embedded Systems, in Proceedings of the ACM SIGPLAN 2003 conference on Programming language design and implementation.

² AVR Microcontrollers for High-Performance and Power-Efficient 8-bit Processing. Atmel- 45058A-About-AVR-Article_US_092013.

³ Femto OS. Web page. Visited 2016-11-07. <http://www.femtoos.org>.

rich connectivity protocol stack for Bluetooth Low Energy (BLE), Ethernet, Thread, Wi-Fi, ZigBee IP, ZigBee NAN, 6LoWPAN. Moreover, some unique features of mbed OS such as automatic power management, device, and software protection, compatibility with the transport layer security protocols (i.e. TLS, DTLS), and the support for OneM2M standard architecture and protocols make it suitable for RC devices in IoT applications.⁴

5.4. ARM architecture

The ARM architecture is considered as the most advanced architecture of the modern era. It supports and caters for almost all IoT implementations with excellent performance. Due to its architectural simplicity, the ARM has a tiny footprint and resultantly consumes less power. This little implementation footprint together with high computation power and low power consumption makes ARM a very handy architecture for SIoT applications. It is a RISC architecture, equipped with following typical features:

- (1) Supports direct computations on register contents.⁵
- (2) Simple addressing modes, with all load/store addresses being determined from register contents and instruction fields only.
- (3) Uniform and fixed-length instruction set, to simplify instruction decode.
- (4) Auto-increment and auto-decrement addressing modes to efficiently handle program loops.
- (5) The option of conditional executions of instructions with most mnemonics.
- (6) Instructions with control over both ALU and shifter to maximize their throughput.

The latest versions of the ARM are divided into three main categories, i.e. Cortex-M, Cortex R and Cortex A; each having its own key features. The unique features are separately listed for the three categories and recommendations of suitable OS for these platforms are provided to enable ease of choice in distinct kinds of SIoT applications.

5.4.1. Cortex-M

The salient features of ARM Cortex-M are:

- (1) Hardware Floating Point Unit.
- (2) Powerful DSP extensions.
- (3) Low interrupt latency.
- (4) Memory Protection Unit.
- (5) Instructions for bit manipulations.
- (6) Rich Connectivity Interfaces to provide connectivity for communication modules (Wi-Fi, ZigBee, Bluetooth).⁶

These features make it suitable for SIoT and M2M devices, robotics, mathematical computations, implementation of DSP filters and some complex sensor network nodes. The mbed OS is specifically designed for ARM architecture and thus suitable for all the three categories. Other suitable OS for the Cortex M architecture includes TinyOS, RIOT, FreeRTOS, and Contiki [32], etc. While some of them are already elaborated, the Contiki OS' key features are discussed below:

5.4.1.1. Contiki. The main components in the Contiki OS are the kernel, the libraries, the program loader, and a set of processes.

All of these processes, including both application programs and services, can be dynamically loaded and swapped at run-time. The process state is held in the process' private memory, and the kernel only keeps a pointer to the process state. To satisfy the criteria for IoT devices, the event kernel in Contiki is very lightweight. It dispatches the process to running queue. The event handler cannot be preempted once it has been scheduled. Contiki offers different services which are created by one process, and other processes use these services. A service is like a shared library in Contiki. It can be dynamically swapped at runtime and must, therefore, be dynamically linked. Typical examples include communication protocol stacks, drivers for wireless communication chips, and some APIs implementing sensor data handling algorithms [23]. Moreover, on top of the event-based kernel, there is a library for preemptive multithreading in Contiki. The library links to applications on a need basis and explicitly requires a multi-threaded model of operation. The OS footprint is larger than that of TinyOS but occupies less program memory than that of the Mantis system. Unlike the TinyOS event kernel which only offers a FIFO event scheduler, the Contiki OS kernel also provides support for the priority based poll handlers.

5.4.2. Cortex-R

The salient features of ARM Cortex-R are:

- (1) High performance: rapid execution of complex code and DSP features.
- (2) Real-time: designed to ensure deterministic responsiveness and high throughput.
- (3) Reliable: detects errors and maintains system operation.⁷
- (4) Configurable: Configuration to include/exclude features to attain desired power, performance and area.

The real-time response, fast execution speed, and reliability make this platform suitable for supporting some powerful real-time OSs such as RIOT and RTLinux. Their key features are described to illustrate how they utilize the Cortex-R platform.

5.4.2.1. RIOT. RIOT is a light-weight operating system for networked, memory-constrained systems with a focus on low-power wireless IoT devices. RIOT is based on microkernel architecture supporting multithreading. It contains some powerful libraries and TCP/IP network stacks. It is robust against bugs because of its modular architecture. If an error occurs in a module, then rest of modules are unaffected. The kernel constantly allocates time-periods to scheduled tasks for real-time response. It achieves a constant runtime of the scheduler by using a fixed-sized circular linked list of threads. The deterministic response of timer operations is obtained using compare registers. OS based auto deep sleep modes are provided in order to implement energy-efficiency.

For more powerful IoT devices, RIOT incorporates a non-periodic event-based schedule. If there are no pending tasks, it goes into deep sleep saving energy. The kernel in RIOT provides a light-weight scheduler that is called within an interrupt service routine. This minimizes the process switching time as there is no need to save the old thread context.

5.4.2.2. RTLinux. RTLinux is a hard real-time RTOS based on microkernel architecture. Preemptive scheduling is followed in RTLinux. This deterministic real-time property makes it suitable for applications in robotics, sensor-based data acquisition systems, and other time-sensitive instruments and machines. RTLinux adds the capability of running special time based deterministic tasks on the same machine where all Linux services are already in running

⁴ Mbed IoT Device Platform. Web page. Visited 2016-11-07. <https://www.arm.com/products/internet-of-things-solutions/mbed-iot-device-platform.php>.

⁵ ARM Architecture Reference Manual. ARM DDI 0100I.

⁶ Cortex-M Series. Web page. Visited 2016-12-05. <http://www.arm.com/products/processors/cortexm/index.php?tab=Why+Cortex-M?>

⁷ Cortex-R Series. Web page. Visited 2016-12-05. <http://www.arm.com/products/processors/cortr/index.php?tab=Performance>.

state. RTLinux runs in a virtual environment where the Linux “guest” is given a virtualized interrupt controller and timer, and all other hardware are accessed directly. Microkernel in RTLinux is a modular kernel which provides multiple levels of abstraction. One of the modules is *rtl time* which controls the processor clocks and provides an abstract interface for connecting handlers to locks [52]. Real-time tasks are implemented as modules in RTLinux with the rights to access hardware.

5.4.3. Cortex-A

The salient features of Cortex-A processors are:

- (1) Perform complex computations: run applications required by smartphones and complex nodes.
- (2) Fast response: suitable for servers, high-end computational nodes and enterprise applications where large core clusters are combined for optimal solutions.⁸
- (3) Low-power architecture: enables their all-day usage. It also adds support for the widest mobile App ecosystem.
- (4) Scalable: Cortex-A provides highly scalable and power efficient environment for enterprise applications.

These features are best utilized by Linux and Windows-based OS such as Embedded Linux, Android and Windows 10.

5.4.3.1. Embedded Linux. Embedded Linux OS is the specialized version of desktop Linux with a few dissimilarities. Firstly, Linux kernel is configured differently when tailored for embedded systems. The device drivers and file systems also differ in case of Embedded Linux. In addition, more emphasis is paid to development tools. It includes tailored libraries with a smaller footprint. Moreover, Embedded Linux contains very few multi-user scenarios while the GUI has limited features.

5.4.3.2. AndroidThings. Google's Android is based on a Linux kernel 2.6 and is well suited for smartphones and other touchscreen devices. Every Android application has its own process which is running in a separate Dalvik virtual machine.⁹ Dalvik executable files are optimized for minimal memory footprints.¹⁰ A specialized version for IoT devices is named as *AndroidThings*. The main difference is that *AndroidThings* contains libraries for accessing the hardware which is not present in smartphones, such as UART and SPI modules. This support is provided by a library named *Peripheral I/O API*, and another API named *User Driver* to allow user applications to insert hardware events into Android framework. Some packages like media store and telephony are not included in *AndroidThings*.

5.4.3.3. Windows 10 IoT Core. Windows 10 IoT Core is an edition of Windows 10 designed for RC devices with fairly powerful processing and computational features. It is well suited for IoT gateways, hubs and master modules in WSNs. Application framework and development tools are same as that of other Windows.

Table 2 includes the operational characteristics of hardware architectures that facilitate or improve the interaction capabilities among IoT devices within SIoT applications. The list of parameters considered therein, includes the clock speed, computational support, instruction length, communication interface, memory mapping, signal processing, and interrupt latency. For instance, the execution speed is primarily dependent on the number of clocks required to execute a single instruction. Hence, the clock speed parameter in PIC and 8051 architectures is a relatively slower

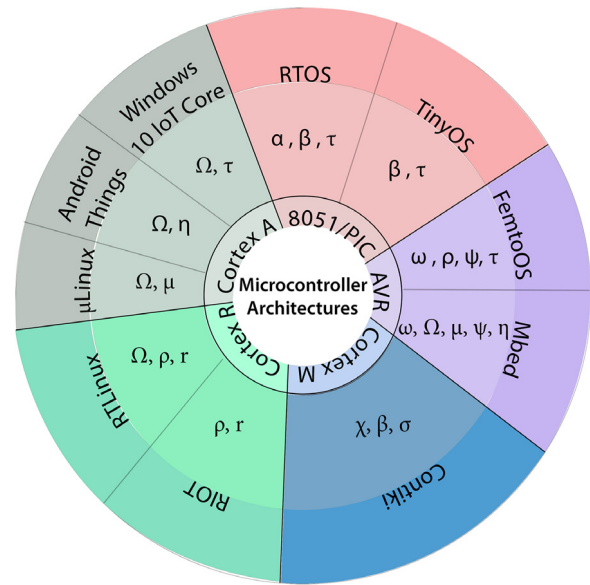


Fig. 2. Unique feature-mapping of OS-to-hardware architectures.

than others. Similarly, cooperation among various devices in SIoT applications usually requires more computations which can be done more precisely and efficiently if corresponding support is available in hardware. The code density is an important factor in deciding the final footprint of the firmware to be run on hardware. The corresponding parameter is instruction length which has a significant effect in achieving the code density. One of the most important parameters is the availability of different communication interfaces on microcontroller as it helps in facilitating the sensor interfacing and interaction with other devices. If the two or more interacting devices have similar hardware architecture, then they can develop a relationship with each other fairly easily. In case of heterogeneous devices with different architectures, we need to consider the corresponding feature support and the availability of different communication interfaces can therefore allow more comfortable device level interaction and relationship establishment. Therefore, the availability of various interfaces within each architecture is also listed in Table 2. Moreover, the overall performance of a hardware architecture is influenced by memory architecture adapted to corresponding hardware. Another relevant parameter to hardware performance is interrupt latency which signifies the number of clock cycles required to go into ISR on receiving an interrupt. In addition, the support for signal processing capabilities of any architecture is also provided in Table 2. Hence, all of these parameters can play a crucial role in enabling various kinds of services in the SIoT application through appropriate interaction and collaboration activities among devices. Accordingly, suitable OSs for each of these architectures are listed at the end of Table 2.

Furthermore, the feature-mapping of different OSs features to that of hardware architectures is graphically summarized in Fig. 2. It is noteworthy to mention here that Fig. 2 is not a quantitative measure of any hardware or an OS performance. In fact, it rather provides the much needful information on qualification of any OS to support the relevant feature within the hardware of an IoT device to boost its interaction possibilities with other devices. Therefore, an OS which has more mapping features mentioned against, does not necessarily mean that it has superior capabilities. For instance, Embedded Linux (μ Linux) just contains two mapping symbols for Cortex A, but it does not imply that it is an inferior OS as compared to Femto OS which has more mapping symbols for AVR. On the other hand, we can certainly argue that any OS

⁸ Cortex-A Series. Web page. Visited 2016-12-15. <http://www.arm.com/products/processors/cortexa/index.php?tab=Multicore>.

⁹ Developers, Android. “What is android”. (2011).

¹⁰ The Android Source Code: Governance Philosophy”. source.android.com. December 17, 2014. Retrieved February 11, 2017.

Table 2

Potentials of hardware architectures for service provisioning in social IoT applications.

Hardware architecture	Clock speed (cycles/instruction)	Compute support	Instruction length (bytes)	Communication interfaces	Memory mapping	Interrupt latency (in no. of clock cycles)	Signal processing	Suitable OSs
8051/PIC	12/4	ALU	1, 2	PIC, UART, USART, LIN, CAN, Ethernet, SPI, I2S, I2C	Von Neumann architecture/Harvard architecture	10–18	ADC, PWM	RTOS, TinyOS
ATMEL	1	ALU	2	UART, SPI, I2C, (special purpose AVR support CAN, USART, USB, Ethernet)	Modified Harvard architecture	10–20	ADC, PWM	FemtoOS, mbed
CORTEX-M	1	ALU+FPU	2, 4	LIN, I2C, SPI, UART, CAN, USART, IrDA, USB, Ethernet, I2S, DSP, SAI (serial audio interface)	Modified Harvard architecture	12–16	ADC, PWM, DSP extensions	Contiki
CORTEX-R	1	ALU+FPU	2, 4	UART, USART, Ethernet, I2S, DSP, SAI (serial audio interface), LIN, I2C, SPI, CAN, USB, IrDA	Modified Harvard architecture	~20	ADC, PWM, DSP extensions	RIOT, RTLinux
CORTEX-A	1	ALU+FPU+HPC (for 64 bit processors)	4, 8	UART, LIN, I2C, SPI, CAN, USB, Ethernet, I2S, USART, DSP, SAI (serial audio interface), IrDA	Modified Harvard architecture	~100	ADC, PWM, DSP extensions, co-processor	Embedded Linux, Android Things, Windows 10 IoT Core

or hardware which contains more mapping symbols is certainly more suitable for typical SIoT application, since most of them have tiny low-end embedded devices with limited resources. For example, mbed is a potential suitable OS for many IoT applications as it has the most number of mapping symbols cite against it in Fig. 2. Moreover, although the figure recommends OSs for a specific hardware architecture based on the provided mapping features. However, these OSs can still run on an entirely different hardware architecture but with less performance gains.

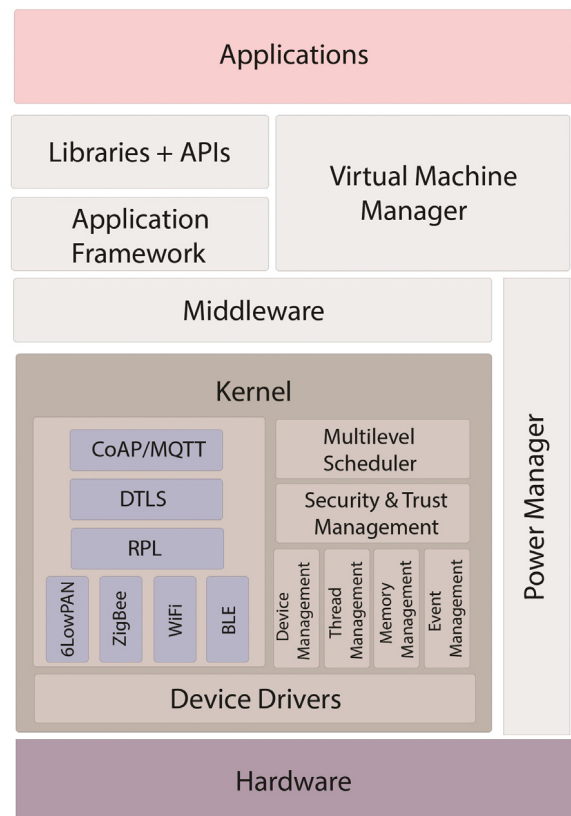
6. Principle requirements in OS for IoT applications

6.1. Mapping of OS and hardware features for IoT use cases

In this section, we first provide the feature-mapping of IoT platforms (both OS and hardware) for various SIoT and/or IoT applications in the form of a Table (Table 3). Thus, the IoT devices employed in these applications exhibit different hardware architectures as well as corresponding OSs, details of which are specified in previous Section. Subsequently, we articulate the necessary and sufficient features for a model OS to meet the requirements of futuristic SIoT applications.

6.2. Required OS features for futuristic SIoT applications

In this sub-section, we propose following recommendations for the model OS for constrained devices in various SIoT applications. Some of these features are available in existing OSs, while others are missing but considered necessary for achieving the optimal performance and better interaction among devices. Based on these recommendations, an OS architecture design is proposed in Fig. 3 which covers all the requirements to be a good candidate OS for SIoT domain. It is pertinent to note that the middleware as well as kernel in Fig. 3, should be both context aware.¹¹

**Fig. 3.** Proposed OS architecture design for SIoT.

- (1) *Modular approach* should be inherently supported in the proposed model OS, wherein modules should be added as per the requirement to reduce the memory footprint. As the nature of interactions in SIoT varies due to the device heterogeneity, therefore it is impossible to make a single

¹¹ Baccelli, Emmanuel, et al. "OS for the IoT-Goals, Challenges, and Solutions". Workshop Interdisciplinaire sur la Sécurité Globale (WISG2013). 2013.

Table 3
Platform recommendations for IoT applications.

IoT applications	IoT use-cases	Feature mapping symbols	Suitable hardware platforms	Suitable OS platforms
Business intelligence	Supply chain & retail management, Logistics and transportation	β, σ, η	PIC, AVR, Cortex-M	RTOS, mbed, Contiki
Multimedia streaming	Video surveillance, Online gaming	$\beta, \sigma, r, \omega, \rho, \chi, \psi, \Omega$	Cortex-A	Embedded Linux, Android Things
Security and Surveillance	Location tracking & monitoring	$\chi, \rho, \tau, \omega, \Omega$	Cortex R, Cortex-M	RTLinux, RIOT, Contiki
Telemedicine	Remote patient monitoring & operation, On-demand doctors	$\sigma, \mu, \omega, \psi, r, \rho, \Omega$	Cortex-A, Cortex-R	Embedded Linux, RTLinux
Smart grids	SCADA applications, automatic pricing	σ, Ω, r, ρ	AVR, Cortex-M	RIOT, RTLinux, Contiki
Smart agriculture	Smart water management, smart farming	β, η, ψ	8051, PIC, AVR	RTOS, TinyOS

customize solution that is fit for all kind of applications. Hence, a highly modular approach is required wherein the services (or protocols) which will be inherently designed to be composable and the new services (or protocols) can be created by combining the existing ones without any undue increase in the power consumption and/or code density. Moreover, unlike TinyOS where the protocol implementations are still vertically designed most of the time, any module (service or a protocol) implementation within the model OS should be independent of other modules with interchangeable libraries and loosely coupled codes and interfaces.

- (2) The model OS should be *event driven*, but unlike Contiki OS, the *support for preempting* the event handler should be available. For instance, in Contiki OS scheduling is done at a single level and events once scheduled are unable to preempt each other. However, this becomes a limitation in the scheduling procedure of Contiki OS as any unscheduled but a top priority event may face a un-necessary delay in the event execution. The preemption capability is specifically crucial in a SIoT application scenario where collaboration among devices is needed but if the support for preemption is not provided by their OSs, it may result in lack of real-time service provisioning.
- (3) Further, to allow a realistic response, kernel level changes should be incorporated in the model OS with built-in smartness and with the support to preempt the already scheduled tasks. Thus, as the higher priority tasks do not get interrupted normally but in the context of SIoT application an accidental event may require an immediate response, thus the OS routines should be *context-aware* to allow runtime changes in tasks' schedule and their interrupt priorities. Henceforth, we consider that the *context awareness* should be inherently incorporated within the *kernel and middleware* of the proposed model OS.
- (4) Lightweight *memory management* and *memory sharing algorithms* as well as *caching techniques* for SIoT nodes should also be incorporated. As the number of collaborating nodes in social IoT application directly affect the memory consumption, therefore memory sharing and caching techniques are crucial not only in reducing the burden of computation on constrained devices but also in improving the associated latency. Moreover, existing memory management algorithms are typically designed for desktop applications. So, there is a dire need for lightweight memory management algorithms which not only ensures the efficient usage of memory but also consumes less number of CPU bursts.
- (5) To enhance the efficiency of scheduling algorithms, *multi-level scheduling* should be implemented in the OS which

is also helpful in avoiding the deadlocks in memory constrained environments. By implementing multi-level scheduling in the OS, we can achieve better interaction among devices in SIoT application particularly in facilitating the real-time service discovery while accessing the common storage locations. However, due to constrained nature of some of the devices in SIoT applications, one needs to be careful not to overburden the devices with intensive computations.

- (6) The *communication stack* of proposed model OS should be flexible enough to support the rich type of low power connectivity interfaces, such as BLE, ZigBee, NFC, and low power Wi-Fi, to provide the opportunities for socialization among a variety of devices in SIoT applications. Therefore, built-in drivers for these radio technologies should be supported to facilitate the IoT application developers.
- (7) Further, the support for various *networking, application layer and security protocols* in communication stack is another primary requirement which in turn helps in improving the collaboration among heterogeneous devices in SIoT applications. Suitable protocols for constrained IoT devices (such as 802.15.4, RPL, DTLS, MQTT, etc.) should be available in a customizable fashion so that any type of additions and omissions to any specific functionality can be done seamlessly without affecting the main purpose of a protocol.
- (8) *Virtualization* concept should be integrated into the OS so that each virtual machine supports at least one communication stack with the flexibility of adding another one dynamically, when dissimilar interface connectivity is required. It can also assist SIoT devices and SIoT server in service composition as well as in relationship management. For this reason, Virtual Machine Manager is specifically emphasized in Fig. 3.
- (9) Correlated to all the features defined above, one of the most essential aspects that should be considered in the design and development of OS is to keep its *footprint* to a viable minimum. Since most of the IoT devices have tiny memory options, it should not exceed the memory limitations imposed by the hardware. The small OS footprint can be achieved by adapting the modular approach (discussed earlier) and by using the *compact combination of instructions* provided by the CPU. In addition, fast execution speed is another primary requirement which is affected by the nature of the combination of instructions used.

We consider that all these above-mentioned features are essential in order to fulfill the requirements of existing as well as futuristic SIoT applications. The model OS thus proposed, can play a vital role in facilitating the establishment of different kinds of relationships among interacting devices according to the nature of various application scenarios. Finally, a taxonomy of these features

Table 4

Taxonomy of proposed features availability in existing OS.

Proposed OS features	Contiki	TinyOS	Free RTOS	mbed	RIOT	AndroidThings
Real-time response	×	×	✓	×	✓	×
Modular Architecture	×	✓	×	✓	✓	×
Event Driven Kernel	✓	✓	×	×	×	×
Context-Aware Kernel	×	×	×	×	×	×
Rich Communication Stack	✓	×	×	✓	×	✓
IoT related protocols at different layers	✓	×	×	✓	×	✓
Virtualization	✓	×	×	×	×	✓
Small Footprint	✓	✓	✓	✓	✓	×
Fast Execution Speed	×	✓	✓	✓	✓	×
Memory Management Algorithms	×	✓	×	×	×	✓
Multi-level Scheduling	✓	×	×	×	×	✓

in currently available OSs is shown in Table 4. It includes the proposed OS features and presents a taxonomy of their availability in the existing operating systems which are typically used in IoT devices.

7. Open research challenges

In this Section, we highlight the most significant challenges in realizing the proposed features in model OS. We consider that the development of the model OS as proposed in the previous Section has to face several research and development challenges due to the memory requirements, hardware limitations on which the OS would run and heterogeneous necessities of SloT applications. Hence, this Section provides research directions to the developers and domain researchers to investigate and develop a full-stack OS to meet the demand of futuristic SloT applications.

7.1. Optimized resource utilization

The available resources are limited in an embedded environment as compared to traditional OS and hence their optimization is necessary. The hardware resource dependencies for the required OS should also be minimized. Thus, guaranteed scheduling in a distributive manner at various levels is a crucial challenge. Solving this multi-objective function of optimal resource allocation is an open research direction due to diverse nature of SloT applications and their varying user requirements.

7.2. Security and trust management

Security has always been a fundamental challenge in the widespread adaptation of any technology. In case of SloT, the desire for trust establishment become even more crucial to develop any mutual relationship among two or more IoT devices. The security protocols within OS should be made robust as well as flexible enough to ensure that the authentication and trust management tasks are carried out efficiently in various SloT application scenarios. Different kinds of protocols and interfaces running on OS may require a distinct type of encryption algorithms. These algorithms, however, need to be computationally less intensive and should be incorporated while considering the memory and latency requirements. In addition, these algorithms should run in real time to avoid any performance degradation. Correspondingly, the memory handling can also raise the security concerns.

7.3. Compatibility

Another challenge in SloT applications is the compatibility of heterogeneous devices from different vendors which may need to interoperate with each other [53]. Rich communication stack provides this facilitation. However, to ensure connectivity, coherent performance and to address interdependency, the OS should offer uniform standardized interfaces independent of language and thus compatibility barriers/issues. The collaborative inter-

faces will allow developers to design and develop various services accordingly.

7.4. Accuracy and reliability

While dealing with delay sensitive SloT applications, where a process completion time is merely order of nanoseconds, the model OS needs to adhere to stringent requirements which is pivotal for the safety IoT devices and machines as well as their users. The mission critical services and applications [54] in which different devices collaborate, such as a vehicle to vehicle communication, and live operation monitoring, necessitate that the performance metrics like latency, reliability, etc., are aptly ensured. Thus the model OS should allow the development of highly precise systems.

7.5. I/O handling

The management of I/O system calls within an OS is imperative for various IoT and SloT applications. Lack of well-defined I/O management may lead to deadlocks, inefficient resource utilization, delays and also vulnerability to security threats. An associated challenge in I/O management is compatibility since the I/O handler should be device independent software due to the heterogeneity of various SloT devices.

7.6. Privileged instructions

Another challenge faced when virtual machines are created inside an OS is to provide support for running privileged instructions in the OS. The challenge may arise while generating virtual instruction against privileged instructions. Moreover, the effect of defining system calls (virtual commands) should not be visible on overall OS footprint.

7.7. Cost effectiveness

The efficient and profitable service development and provisioning on OS regarding energy or compute cost is a significant challenge in SloT applications [11]. Multiple processes can run simultaneously which may change as per variations in applications and user requirements. In SloT, IoT devices render different kind of services to improve and while many of them are battery powered some requiring battery life up to ten years, so duty cycling is challenging due to their varying nature.

8. Conclusion

This research work describes the key features of the latest available resource constrained hardware platforms for SloT applications. Based on these features, specific OSs are recommended for each platform to utilize their hardware capabilities efficiently. Structure, architecture and operational features of recommended

OSs are described that make efficient use of hardware architectural potentials of devices employed in SIoT applications. Therefore, a comparative study is presented on features of OSs and hardware architectures particularly focused on resource-constrained devices. While enabling these necessary features in OS architecture, associated challenges that may arise and should be taken into account are also articulated in this paper. Thus, the study opens doors for future work since following the proposed recommendations, more robust algorithms can be implemented to better utilize the specific hardware architectural features of devices in SIoT applications. Moreover, this research can be taken as initiating guidelines for developers to design an efficient OS to meet the demand of futuristic IoT and SIoT applications.

References

- [1] Gubbi Jayavardhana, et al., Internet of Things (IoT): A vision architectural elements and future directions, *Future Gener. Comput. Syst.* 29.7 (2013) 1645–1660.
- [2] Yaqoob Ibrar, et al., Internet of things architecture: Recent advances, taxonomy, requirements, and open challenges, *IEEE Wirel. Commun.* 24.3 (2017) 10–16.
- [3] Ejaz Ahmed, and Mubashir Husain Rehmani, Introduction to the Special Section on Social Collaborative Internet of Things, 2017, pp. 382–384.
- [4] Khan Wazir Zada, et al., When social objects collaborate: Concepts processing elements attacks and challenges, *Comput. Electr. Eng.* 58 (2017) 397–411.
- [5] Baskiyar Sanjeev, Natarajan Meghanathan, A survey of contemporary real-time operating systems, *Informatica (Slovenia)* 29.2 (2005) 233–240.
- [6] Tor-Morten Grønli, Jarle Hansen and Gheorghita Hinea, Android vs Windows Mobile vs Java ME: A comparative study of mobile development environments, in: *Proceedings of the 3rd International Conference on Pervasive Technologies Related to Assistive Environments*, Article No. 45.
- [7] Chandra Tej Bahadur, Pushpak Verma, A.K. Dwivedi, Operating systems for internet of things: A comparative study, in: *Proceedings of the Second International Conference on Information and Communication Technology for Competitive Strategies*, ACM, 2016.
- [8] Rixner, Scott, et al., Memory access scheduling, *ACM SIGARCH Comput. Architect. News* 28 (2) (2000) ACM.
- [9] Mutlu Onur, Thomas Moscibroda, Stall-time fair memory access scheduling for chip multiprocessors, in: *Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture*, IEEE Computer Society, 2007.
- [10] Gerstlauer Andreas, Haobo Yu, Daniel D. Gajski, RTOS modeling for system level design, in: *Embedded Software for SoC*, Springer, US, 2003, pp. 55–68.
- [11] Huang Chung-Ming, et al., The social internet of thing (S-IOT)-based mobile group handoff architecture and schemes for proximity service, *IEEE Trans. Emerg. Top. Comput.* (2017).
- [12] Morais Raul, A. Valente, and C. Serôdio. A wireless sensor network for smart irrigation and environmental monitoring: A position article, in: *5th European federation for information technology in agriculture, food and environment and 3rd world congress on computers in agriculture and natural resources*, EFITA/WCCA, 2005.
- [13] Kim Hyoseung, Hojung Cha, Multithreading optimization techniques for sensor network operating systems, in: *European Conference on Wireless Sensor Networks*, Springer, Berlin, Heidelberg, 2007.
- [14] Dong Wei, et al., Providing OS support for wireless sensor networks: Challenges and approaches, *IEEE Commun. Surveys Tutor.* 12.4 (2010) 519–530.
- [15] M. Salem Tarek, S.M. Abd El-kader, Mapping wireless sensor network applications requirements to existing operating systems, *Int. J. Comput. Sci. Issues (IJCSI)* 10.5 (2013).
- [16] Wright Chris, et al., Linux security modules: General security support for the Linux kernel, in: *USENIX Security Symposium*, Vol. 2, 2002.
- [17] D. McCullough, uCLinux for Linux Programmers, *Linux J.* (2004).
- [18] V. Reddy, et al., Operating systems for wireless sensor networks: A survey, *Int. J. Sens. Netw.* 5 (2009) 236–255.
- [19] Cha Hojung, et al., The RETOS operating system: Kernel, tools and applications, in: *Proceedings of the 6th international conference on Information processing in sensor networks*, ACM, 2007.
- [20] Cha Hojung, et al., RETOS: Resilient, expandable, and threaded operating system for wireless sensor networks, in: *Information Processing in Sensor Networks*, 2007, IPSN 2007, 6th International Symposium on IEEE, 2007.
- [21] Muhammad Omer Farooq, Thomas Kunz, Operating systems for wireless sensor networks: A survey, *Sensors* 11 (6) (2011).
- [22] Levis Philip, et al., TinyOS: An operating system for sensor networks, in: *Ambient Intelligence*, Springer, Berlin, Heidelberg, 2005, pp. 115–148.
- [23] Adam Dunkels, et al., Contiki - A lightweight and flexible operating system for tiny networked sensors, in: *Local Computer Networks*, 2004. 29th Annual IEEE International Conference.
- [24] Y. Yorozu, et al., A real-time kernel for wireless sensor networks employed in rescue scenarios, in: *Proceedings of the IEEE 34th Conference on Local Computer Networks (LCN)*, October 2009.
- [25] Dunkels Adam, Rime—a lightweight layered communication stack for sensor networks, in: *Proceedings of the European Conference on Wireless Sensor Networks, EWSN, Poster/Demo session*, Delft, The Netherlands, 2007.
- [26] Lettner Michael, Michael Tschernuth, Rene Mayrhofer, Mobile platform architecture review: Android, iphone, qt, in: *International Conference on Computer Aided Systems Theory*, Springer, Berlin, Heidelberg, 2011.
- [27] Baccelli Emmanuel, et al., OS for the IoT—Goals, Challenges, and Solutions, in: *Workshop Interdisciplinaires sur la Sécurité Globale, WISG2013*, 2013.
- [28] Josyula, Surya Kant, and Daya Gupta, Internet of things and cloud interoperability application based on Android. *Advances in Computer Applications*, ICACA, in: *IEEE International Conference on IEEE*, 2016.
- [29] Milinković Aleksandar, Stevan Milinković, Ljubomir Lazić, Choosing the right RTOS for IoT platform, 2015.
- [30] Gaur Padmini, Mohit P. Tahiliani, Operating systems for IoT devices: A critical survey, in: *Region 10 Symposium (TENSYP)*, 2015 IEEE, IEEE, 2015.
- [31] D. Guinard, M. Fischer, V. Trifa, Sharing using social networks in a composable web of things, in: *Proc. of IEEE Percom 2010*, March–April 2010.
- [32] Borgohain Tuhin, Uday Kumar, and Sugata Sanyal, Survey of operating systems for the IoT environment, *arXiv preprint arXiv:1504.02517*, 2015.
- [33] Emmanuel Baccelli, et al., RIOT OS: Towards an OS for the Internet of Things, in: *Proc. of the 32nd IEEE INFOCOM Poster Session*, IEEE Press, Piscataway, NJ, USA, 2013, pp. 2453–2454.
- [34] Abdelsamea Mahmoud Hussein Abdelkarem, Mohamed Zorkany, and Neamat Abdelkader, Real time operating systems for the internet of things, vision, architecture and research directions, in: *Computer Applications & Research, WSCAR*, 2016 World Symposium on IEEE, 2016.
- [35] P.P. Ray, A survey on internet of things architectures, *J. King Saud Univ.-Comput. Inform. Sci.* (2016).
- [36] Hahm Oliver, et al., Operating systems for low-end devices in the internet of things: A survey, *IEEE Internet of Things J.* 3.5 (2016) 720–734.
- [37] Atzori Luigi, Antonio Iera, Giacomo Morabito, From “smart objects” to “social objects”: The next evolutionary step of the internet of things, *IEEE Commun. Mag.* 52.1 (2014) 97–105.
- [38] Ejaz Ahmed, Ibrar Yaqoob, Abdullah Gani, Muhammad Imran, Mohsen Guizani, Social-aware resource allocation and optimization for D2D communication, *IEEE Wireless Commun.* 24 (3) (2017) 122–129.
- [39] Kranz Luis Roalter, F. Michahelles, Things that twitter: Social networks and the Internet of things, in: *Proc. of the Pervasive 2010, the Citizen Internet of Things 2010 workshop CIoT 2010: What Can the Internet of Things Do for the Citizen?* May 2010.
- [40] Atzori Luigi, Antonio Iera, Giacomo Morabito, Siot: Giving a social structure to the internet of things, *IEEE Commun. Lett.* 15.11 (2011) 1193–1195.
- [41] Atzori Luigi, et al., The social internet of things (siot)—when social networks meet the internet of things: Concept, architecture and network characterization, *Computer Netw.* 56.16 (2012) 3594–3608.
- [42] Saleem Yasir, et al., Exploitation of social IoT for recommendation services, *Internet of Things (WF-IoT)*, in: *2016 IEEE 3rd World Forum on, IEEE*, 2016.
- [43] Nitti Michele, Luigi Atzori, Irena Pletikosa Cvijikj, Network navigability in the social internet of things, *Internet of Things (WF-IoT)*, in: *2014 IEEE World Forum on IEEE*, 2014.
- [44] Bao Fenye, Ray Chen, and Jia Guo, Scalable, adaptive and survivable trust management for community of interest based internet of things systems, *Autonomous Decentralized Systems, ISADS*, in: *2013 IEEE Eleventh International Symposium on IEEE*, 2013.
- [45] Chen Ray, Fenye Bao, Jia Guo, Trust-based service management for social internet of things systems, *IEEE Trans. Dependable Secure Comput.* 13.6 (2016) 684–696.
- [46] Atzori Luigi, Davide Carboni, Antonio Iera, Smart things in the social loop: Paradigms technologies and potentials, *Ad Hoc Netw.* 18 (2014) 121–132.
- [47] M. Ortiz Antonio, et al., The cluster between internet of things and social networks: Review and research challenges, *IEEE Internet Things J.* 1.3 (2014) 206–215.
- [48] Girau Roberto, Michele Nitti, Luigi Atzori, Implementation of an experimental platform for the social internet of things, *Innovative Mobile and Internet Services in Ubiquitous Computing, IMIS*, in: *2013 Seventh International Conference on IEEE*, 2013.
- [49] Asl Hamid Zargari, et al., How often social objects meet each other? Analysis of the properties of a social network of IoT devices based on real data, in: *Global Communications Conference, GLOBECOM*, 2013 IEEE, IEEE, 2013.
- [50] Yang Geng, et al., A health-iot platform based on the integration of intelligent packaging unobtrusive bio-sensor and intelligent medicine box, *IEEE Trans. Indust. Inform.* 10.4 (2014) 2180–2191.
- [51] M.A. Mazidi, et al., *The 8051 Microcontroller and Embedded Systems: Using Assembly and C*. Vol. 626, Pearson/Prentice Hall, 2006.
- [52] Yodaiken Victor, Michael Barabanov, RT-Linux. White paper, Department of Computer Science, New Mexico Institute of Technology, Available at <http://www.rtlinux.org/documents>, 2001.
- [53] Ejaz Ahmed, Ibrar Yaqoob, Abdullah Gani, Muhammad Imran, Mohsen Guizani, Internet-of-things-based smart environments: State of the art taxonomy and open research challenges, *IEEE Wirel. Commun.* 23 (5) (2016) 10–16.
- [54] Alam Kazi Masudul, Mukesh Saini, Abdulmotaleb El Saddik, Toward social internet of vehicles: Concept, architecture, and applications, *IEEE Access* 3 (2015) 343–357.



Bilal Afzal is a graduate from National University of Science and Technology (NUST) Islamabad, Pakistan. He has worked as Research Officer and Sr. Research Officer at Al-Khwarizmi Institute of Computer Science (KICS), University of Engineering and Technology (UET) Lahore, Pakistan. Currently, he is working as Sr. Research Officer at Sultan Qaboos IT Chair, UET Lahore. His research interests include Computer and Wireless Networks, Internet of Things, machine to machine (M2M) communication, 5G, and Next Generation Networks. He has served as the Co-Chair and Session Manager in *IEEE ICOSST 2014, 2015, 2017* conferences. He is the regular reviewer of various impact factor journals including *Elsevier FGCS, JNCA, COMPELECENG*, etc., *IEEE Network Magazine, IEEE IoT Journal, SAGE IJDSN*, and several reputed IEEE conferences.



Muhammad Umair completed his B.Sc. Electrical Engineering and M.Sc. Electrical Engineering from University of Engineering & Technology (UET) Lahore in 2014 and 2017, respectively. He worked as a Research Officer in Internet of Things (IoT) lab at Al-Khwarizmi Institute of Computer Sciences, UET Lahore. Recently, he has joined Sultan Qaboos IT research lab as a research officer. His area of interests include Internet of Things, Embedded Systems, Network Systems and Machine Learning.



Ghalib A. Shah [M'09] received his Ph.D. degree in computer engineering from Middle East Technical University, Turkey in 2007. He is currently a Professor and Sultan Qaboos IT (SQIT) Co-Chair, at UET Lahore and leading the research center of SQIT. He has been awarded many research grants by national and international funding organizations. He is a regular technical committee member of many reputed international conferences and reviewer of various high impact factor journals. His research interests include the design and analysis of communication protocols from MAC to Transport layer for cognitive radio networks, wireless multimedia networks and Internet of things.



Ejaz Ahmed received his Ph.D. from University of Malaya, Malaysia. He has successfully published his research work in more than thirty five reputable international journals and conferences. His areas of interest include Mobile Cloud Computing, Mobile Edge Computing, Internet of Things, and Cognitive Radio Networks. Dr. Ejaz is editor in *IEEE Communication Magazine, IEEE Access, Elsevier JNCA and KSII TIS*. He is also serving as a Lead Guest Editor in *Elsevier Future Generation Computer Systems, IEEE Access, Elsevier Computers & Electrical Engineering, IEEE Communication Magazine, Elsevier Information Systems and Transactions on Emerging Telecommunications Technologies (ETT)*.